

# Automated Vulnerability Discovery & Remediation Pipeline (WordPress + WPScan + GitHub Actions)

## 1. Scenario

You are working as a **Security Engineer** in a DevSecOps team.

Your task is to automate part of the vulnerability discovery process for a containerized WordPress deployment.

You will deploy a WordPress environment, scan it using WPScan, remediate vulnerabilities through patching and basic hardening, rebuild the image, and verify fixes through automated rescanning.

*This lab simulates a real-world DevSecOps workflow.*

## 2. Your Objectives

You will:

- Deploy a WordPress instance using **docker-compose**
- Automate vulnerability scanning using **WPScan** in a GitHub Actions workflow
- Analyze **vulnerabilities** and explain potential exploitation paths
- **Patch and update** WordPress core, plugins, themes, and base image
- Apply **basic security hardening** ((e.g. non-root user, remove unnecessary packages, secure config etc.)
- Rebuild and push the fixed image to **GitHub Container Registry** and **Docker Hub**
- Trigger the workflow to **re-scan the fixed version** (semi-automated loop); Manual work counts as well but semi-automatic is preferred.
- Produce a professional **PDF report** summarizing your work

## 3. Required Deliverables

### A. GitHub Repository

Your repo must include:

- /docker/docker-compose.yml (your deployment definition)

- .github/workflows/scan.yml (your automated scanning workflow)
- WPScan output artifacts stored under /scans/
- Fixed/hardened image build files
- Clear commit history showing remediation steps

## **B. Docker Hub Repository**

- Final hardened WordPress image (required)
- Optional: initial vulnerable image

## **C. PDF Report → presentation**

Your report must include:

### **1. Environment Setup**

- Steps taken
- Challenges encountered
- How you solved them

### **2. Findings Overview**

- Key vulnerabilities
- Risk assessment
- How attackers could exploit them

### **3. Remediation Steps**

- Patching and updating
- Hardening measures
- Before/after scan evidence

### **4. Fixed Image Build**

- Link to GitHub repo
- Link to Docker Hub image

### **5. Tooling Justification**

- Why Docker, WPScan, GitHub Actions, etc.

### **6. DevSecOps Strategy**

- How your workflow demonstrates **shift-left security**

#### **4. Tools You Will Use**

- Docker
- WordPress official image
- WPScan
- GitHub Actions
- GitHub Container Registry
- Docker Hub

#### **5. Workflow Summary**

1. Deploy WordPress locally
2. Run WPScan manually
3. Build GitHub Actions workflow to automate scanning
4. Analyze vulnerabilities
5. Apply patches + hardening
6. Rebuild and push fixed image
7. Trigger workflow to re-scan
8. Document everything in your report

Example of git content:



Example of starting workflow:

name: WordPress WPScan – **beware of the miss-configs!!!**

#####

on:

push:  
 branches:  
 - main

jobs:

wordpress-wpscan:  
 runs-on: ubuntu-latest

steps:

- name: Checkout (required even if no repo code is used)  
 uses: actions/checkout@v4
- name: Set up Docker environment  
 run: |  
 sudo systemctl start docker  
 docker version
- name: Start MySQL container

```

run: |
    docker run -d --name wp-db \
        -e MYSQL_ROOT_PASSWORD=password \
        -e MYSQL_DATABASE=wordpress \
        -e MYSQL_USER=wpuser \
        -e MYSQL_PASSWORD=wppass \
        -p 3306:3306 \
        mysql:5.7

- name: Start WordPress container
run: |
    docker run -d --name wp-site \
        --link wp-db:mysql \
        -e WORDPRESS_DB_HOST=host.docker.internal:3306 \
        -e WORDPRESS_DB_USER=wpuser \
        -e WORDPRESS_DB_PASSWORD=wppass \
        -e WORDPRESS_DB_NAME=wordpress \
        -p 8080:80 \
        wordpress:latest

- name: Wait for WordPress to be ready
run: |
    echo "Waiting for WordPress to start..."
    for i in {1..30}; do
        if curl -s http://localhost:8080/wp-content/ | grep -q "<title>";
then
            echo "WordPress is ready!"
            break
        fi
        sleep 5
    done

- name: Install Ruby and WPScan
run: |
    sudo apt-get update
    sudo apt-get install -y ruby-full build-essential libcurl4-openssl-
dev zlib1g-dev
    sudo gem install wpscan

- name: Run WPScan
run: |
    wpscan --url http://localhost:8080 --enumerate u,vp,vt --force ||
true

#####

```